Project: Investigate a Dataset - Movie Profitability (TMDb)

Table of Contents

- Introduction
- Data Wrangling
- Exploratory Data Analysis
- Conclusions

Introduction

Dataset Description

This project uses the TMDb movie dataset. The dataset contains information for movies released from 1960 to 2015. It includes movie details such as title, release year, genre, runtime, popularity, user ratings, vote counts, budget, revenue, and inflation-adjusted budget and revenue.

For this project, I'm focusing on movie profitability. Since the movies were released across many decades, I'll use the inflation-adjusted budget and revenue columns to calculate profit, making older and newer movies easier to compare.

The main dependent variable for this analysis is:

`profit_adj = revenue_adj - budget_adj`

This means adjusted profit is calculated as adjusted revenue minus adjusted budget.

Column Description

- `id` : Unique TMDb movie ID.
- `imdb_id` : Movie ID from IMDb.
- `popularity` : TMDb popularity score.
- `budget` : Original movie budget.
- `revenue` : Original movie revenue.
- `original_title` : Movie title.
- `cast` : Main cast members.
- `homepage` : Movie website, when available.
- `director` : Movie director.
- `tagline` : Movie tagline.

- `keywords` : Keywords connected to the movie.
- `overview` : Short movie description.
- `runtime` : Movie length in minutes.
- `genres` : Movie genre or genres.
- `production_companies` : Companies that produced the movie.
- `release_date` : Movie release date.
- `vote_count` : Number of TMDb user votes.
- `vote_average` : Average TMDb user rating.
- `release_year` : Year the movie was released.
- `budget_adj` : Budget adjusted for inflation.
- `revenue_adj` : Revenue adjusted for inflation.

Questions for Analysis

Which movie characteristics are associated with higher adjusted profit?

1. How is adjusted profit distributed across the movies with complete financial data?
2. Do higher adjusted budgets tend to relate to higher adjusted profits?
3. How do popularity, vote count, and vote average relate to adjusted profit?

In [1]:
```
# Import packages for the project
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline

# Make larger tables easier to read in the notebook
pd.set_option('display.max_columns', 50)
pd.set_option('display.float_format', '{:,.2f}'.format)
```

Data Wrangling

In this section, I'll load the dataset, inspect its structure, check for missing values and duplicates, and identify values that need cleaning before analysis.

In [2]:
```
# Load the TMDb movie dataset
movies = pd.read_csv('tmdb-movies.csv')

# Display the first few rows
movies.head()
```

Out[2]:

	id	imdb_id	popularity	budget	revenue	original_title	
0	135397	tt0369610	32.99	150000000	1513528810	Jurassic World	Chris Pratt|B D Howard|I Khan
1	76341	tt1392190	28.42	150000000	378436354	Mad Max: Fury Road	Hardy|Cha Theron|H Ke Byrne|N
2	262500	tt2908446	13.11	110000000	295238201	Insurgent	Shai Woodley|T James| Winslet|An
3	140607	tt2488496	11.17	200000000	2068178225	Star Wars: The Force Awakens	Harr Ford|M Hamill|C Fisher|Adam
4	168259	tt2820852	9.34	190000000	1506249360	Furious 7	Vin Diesel| Walker|Ja Statham|Mic

In [3]:

```
# Check the shape of the dataset
movies.shape
```

Out[3]:

```
(10866, 21)
```

In [4]:

```
# Check column names, data types, and non-null values
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10866 entries, 0 to 10865
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   id                    10866 non-null  int64
 1   imdb_id               10856 non-null  object
 2   popularity            10866 non-null  float64
 3   budget                10866 non-null  int64
 4   revenue               10866 non-null  int64
 5   original_title        10866 non-null  object
 6   cast                  10790 non-null  object
 7   homepage              2936 non-null   object
 8   director              10822 non-null  object
 9   tagline               8042 non-null   object
 10  keywords              9373 non-null   object
 11  overview              10862 non-null  object
 12  runtime               10866 non-null  int64
 13  genres                10843 non-null  object
 14  production_companies  9836 non-null   object
 15  release_date          10866 non-null  object
 16  vote_count            10866 non-null  int64
 17  vote_average          10866 non-null  float64
 18  release_year          10866 non-null  int64
 19  budget_adj            10866 non-null  float64
 20  revenue_adj           10866 non-null  float64
dtypes: float64(4), int64(6), object(11)
memory usage: 1.7+ MB
```

In [5]:
```
movies.describe()
```

Out[5]:

	id	popularity	budget	revenue	runtime	vote_count
count	10,866.00	10,866.00	10,866.00	10,866.00	10,866.00	10,866.00
mean	66,064.18	0.65	14,625,701.09	39,823,319.79	102.07	217.39
std	92,130.14	1.00	30,913,213.83	117,003,486.58	31.38	575.62
min	5.00	0.00	0.00	0.00	0.00	10.00
25%	10,596.25	0.21	0.00	0.00	90.00	17.00
50%	20,669.00	0.38	0.00	0.00	99.00	38.00
75%	75,610.00	0.71	15,000,000.00	24,000,000.00	111.00	145.75
max	417,859.00	32.99	425,000,000.00	2,781,505,847.00	900.00	9,767.00

In [6]:
```
# Count missing values by column
movies.isna().sum().sort_values(ascending=False)
```

Out[6]:
```
homepage                7930
tagline                 2824
keywords                1493
production_companies    1030
cast                      76
director                  44
genres                    23
imdb_id                   10
overview                   4
budget_adj                 0
release_year               0
vote_average               0
vote_count                 0
release_date               0
id                         0
runtime                    0
original_title             0
revenue                    0
budget                     0
popularity                 0
revenue_adj                0
dtype: int64
```

In [7]:
```
# Check for duplicate rows
movies.duplicated().sum()
```

Out[7]:
```
1
```

In [8]:
```
# Check for zero values in columns where zero is likely missing or unusable
zero_check_columns = ['budget', 'revenue', 'runtime', 'budget_adj', 'revenue
(movies[zero_check_columns] == 0).sum()
```

Out[8]:
```
budget         5696
revenue        6016
runtime          31
budget_adj     5696
revenue_adj    6016
dtype: int64
```

Data Wrangling Findings

The original dataset contains 10,866 rows and 21 columns. Several columns have missing values, especially `homepage`, `tagline`, `keywords`, and `production_companies`.

There's one duplicate row, and many rows have zero values for `budget`, `revenue`, `budget_adj`, or `revenue_adj`. For a financial analysis, those zero values are a problem because profit can't be calculated correctly if they are missing.

There's also some movies with a runtime of zero. Since movies can't have zero runtime, those rows will be removed.

Data Cleaning

For this analysis, I'll will clean the data in these steps:

1. Remove duplicate rows.
2. Keep only the columns needed.
3. Remove rows with missing genre values.
4. Remove rows with zero runtime.
5. Create a separate profit-focused dataframe using only rows where `budget_adj` and `revenue_adj` are both greater than zero.
6. Create the new columns `profit_adj`, and `profit`.
7. Create easier-to-read columns in millions for `budget`, `revenue`, and `adjusted profit`.

In [9]:
```
# Make a copy so the original dataframe stays unchanged
movies_clean = movies.copy()

# Remove duplicate rows
movies_clean = movies_clean.drop_duplicates()

# Keep only the columns needed for the analysis
columns_to_keep = [
    'id', 'original_title', 'popularity', 'budget', 'revenue', 'runtime',
    'genres', 'vote_count', 'vote_average', 'release_year',
    'budget_adj', 'revenue_adj']
movies_clean = movies_clean[columns_to_keep]

# Remove rows with missing genre values and zero runtime
movies_clean = movies_clean.dropna(subset=['genres'])
movies_clean = movies_clean.query('runtime > 0')

# Create a profit-focused dataframe.
# Profit requires both adjusted budget and adjusted revenue, so zero values
profit_movies = movies_clean.query('budget_adj > 0 and revenue_adj > 0').cop

# Create profit columns
profit_movies['profit_adj'] = profit_movies['revenue_adj'] - profit_movies['
profit_movies['profit'] = profit_movies['revenue'] - profit_movies['budget']

# Create columns in millions so charts are easier to read
profit_movies['budget_adj_millions'] = profit_movies['budget_adj'] / 1_000_0
profit_movies['revenue_adj_millions'] = profit_movies['revenue_adj'] / 1_000
profit_movies['profit_adj_millions'] = profit_movies['profit_adj'] / 1_000_0

profit_movies.head()
```

Out[9]:

	id	original_title	popularity	budget	revenue	runtime	
0	135397	Jurassic World	32.99	150000000	1513528810	124	Action|Adventu Ficti
1	76341	Mad Max: Fury Road	28.42	150000000	378436354	120	Action|Adventu Ficti
2	262500	Insurgent	13.11	110000000	295238201	119	Adventu Ficti
3	140607	Star Wars: The Force Awakens	11.17	200000000	2068178225	136	Action|Adventu Fictio
4	168259	Furious 7	9.34	190000000	1506249360	137	Action|Cri

In [10]:

```
# Compare the original and cleaned dataframe sizes
cleaning_summary = pd.DataFrame({
    'Dataframe': ['Original dataset', 'After basic cleaning', 'Profit analys
    'Rows': [movies.shape[0], movies_clean.shape[0], profit_movies.shape[0]]
    'Columns': [movies.shape[1], movies_clean.shape[1], profit_movies.shape[
})

cleaning_summary
```

Out[10]:

	Dataframe	Rows	Columns
0	Original dataset	10866	21
1	After basic cleaning	10812	12
2	Profit analysis dataset	3854	17

In [11]:

```
# Confirm the profit analysis dataset is ready for analysis
profit_movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 3854 entries, 0 to 10848
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   id                    3854 non-null   int64
 1   original_title        3854 non-null   object
 2   popularity            3854 non-null   float64
 3   budget                3854 non-null   int64
 4   revenue               3854 non-null   int64
 5   runtime               3854 non-null   int64
 6   genres                3854 non-null   object
 7   vote_count            3854 non-null   int64
 8   vote_average          3854 non-null   float64
 9   release_year          3854 non-null   int64
 10  budget_adj            3854 non-null   float64
 11  revenue_adj           3854 non-null   float64
 12  profit_adj            3854 non-null   float64
 13  profit                3854 non-null   int64
 14  budget_adj_millions   3854 non-null   float64
 15  revenue_adj_millions  3854 non-null   float64
 16  profit_adj_millions   3854 non-null   float64
dtypes: float64(8), int64(7), object(2)
memory usage: 542.0+ KB
```

In [12]:

```
# Summary statistics for the final profit analysis dataset
profit_movies[['budget_adj_millions', 'revenue_adj_millions', 'profit_adj_mi
               'popularity', 'vote_count', 'vote_average', 'runtime', 'relea
```

Out[12]:

	budget_adj_millions	revenue_adj_millions	profit_adj_millions	popularity	vote_
count	3,854.00	3,854.00	3,854.00	3,854.00	3,
mean	44.24	137.06	92.82	1.19	
std	44.81	216.11	194.07	1.48	
min	0.00	0.00	-413.91	0.00	
25%	13.09	18.36	-1.50	0.46	
50%	30.02	61.73	27.37	0.80	
75%	60.61	163.26	107.45	1.37	
max	425.00	2,827.12	2,750.14	32.99	9,

Cleaning Summary

After removing duplicate rows and keeping only the columns needed for this analysis, I created a profit-focused dataframe. Since profit requires both budget and revenue, I removed rows where adjusted budget or adjusted revenue was equal to zero. The final profit analysis dataset contains 3,854 movies.

This smaller dataframe is used for the financial analysis because each row has the information needed to calculate adjusted profit.

Exploratory Data Analysis

This section explores how adjusted profit relates to different movie features. I will use summary statistics and visualizations to look for patterns.

In [13]:
```
# Reusable plotting function for scatter plots.
# This function is used multiple times to avoid repeating the same plotting
def scatter_with_trend(data, x_column, y_column, title, x_label, y_label):
    # Create a scatter plot with a simple linear trend line.
    x_values = data[x_column]
    y_values = data[y_column]

    plt.figure(figsize=(10, 6))
    plt.scatter(x_values, y_values, alpha=0.35)

    # Add a linear trend line using NumPy
    slope, intercept = np.polyfit(x_values, y_values, 1)
    sorted_x = np.sort(x_values)
    plt.plot(sorted_x, slope * sorted_x + intercept, linewidth=2, label='Tre

    plt.title(title)
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()


def summarize_profit_by_category(data, category_column):
    # Summarize adjusted profit by a category column.
    summary = data.groupby(category_column, observed=True).agg(
        movie_count=('id', 'count'),
        mean_profit_millions=('profit_adj_millions', 'mean'),
        median_profit_millions=('profit_adj_millions', 'median'),
        profitable_rate=('profit_adj', lambda x: (x > 0).mean())
    )
    return summary
```

Research Question 1: How is adjusted profit distributed?

Before comparing adjusted profit to other variables, I want to understand the overall spread of movie profits in the cleaned dataset.

In [14]:
```
# Key adjusted profit statistics
profit_summary = profit_movies['profit_adj_millions'].describe()
profitable_percent = (profit_movies['profit_adj'] > 0).mean() * 100
```

```
print(profit_summary)
print(f"Percentage of movies with positive adjusted profit: {profitable_perc
```

```
count   3,854.00
mean       92.82
std       194.07
min      -413.91
25%        -1.50
50%        27.37
75%       107.45
max     2,750.14
Name: profit_adj_millions, dtype: float64
Percentage of movies with positive adjusted profit: 72.1%
```

In [15]:

```
# Use a NumPy array to calculate adjusted profit percentiles
profit_array = profit_movies['profit_adj_millions'].to_numpy()

profit_percentiles = np.percentile(profit_array, [25, 50, 75, 90])

profit_percentile_summary = pd.DataFrame({
    'Percentile': ['25th', '50th / Median', '75th', '90th'],
    'Adjusted Profit (Millions of Dollars)': profit_percentiles
})

profit_percentile_summary
```

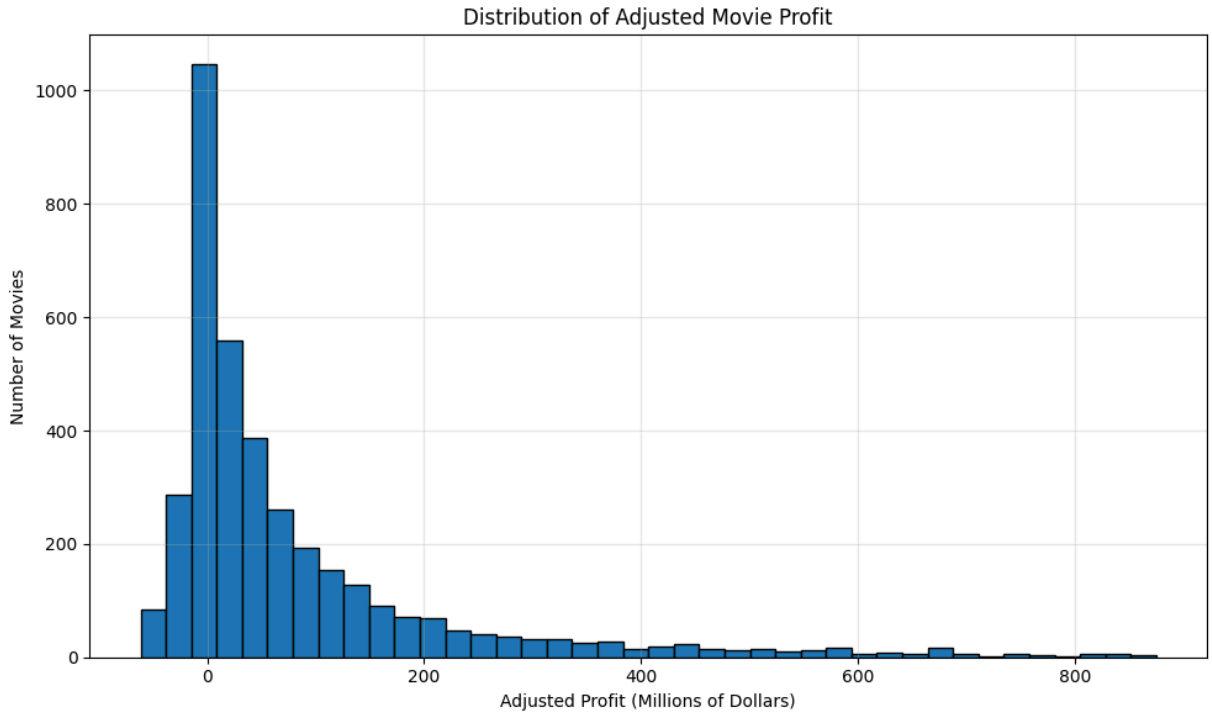
Out[15]:

	Percentile	Adjusted Profit (Millions of Dollars)
0	25th	-1.50
1	50th / Median	27.37
2	75th	107.45
3	90th	275.99

The NumPy percentile calculation shows that adjusted profit is highly spread out. The median movie earned much less than the highest-profit movies, while the 90th percentile shows that only a smaller group of movies earned very high adjusted profits.

In [16]:

```
# Histogram of adjusted profit.
# The x-axis is limited to the 1st and 99th percentiles so the main distribu
lower_limit = profit_movies['profit_adj_millions'].quantile(0.01)
upper_limit = profit_movies['profit_adj_millions'].quantile(0.99)

plt.figure(figsize=(10, 6))
plt.hist(profit_movies['profit_adj_millions'], bins=40, range=(lower_limit,
plt.title('Distribution of Adjusted Movie Profit')
plt.xlabel('Adjusted Profit (Millions of Dollars)')
plt.ylabel('Number of Movies')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



In [17]:
```
# Ten movies with the highest adjusted profit
top_profit_movies = profit_movies.nlargest(10, 'profit_adj')[[
    'original_title', 'release_year', 'budget_adj_millions', 'revenue_adj_mi
    'profit_adj_millions', 'popularity', 'vote_average', 'vote_count', 'runt
]]

top_profit_movies
```

Out[17]:

	original_title	release_year	budget_adj_millions	revenue_adj_millions	profit_a
1329	Star Wars	1977	39.58	2,789.71	
1386	Avatar	2009	240.89	2,827.12	
5231	Titanic	1997	271.69	2,506.41	
10594	The Exorcist	1973	39.29	2,167.32	
9806	Jaws	1975	28.36	1,907.01	
8889	E.T. the Extra-Terrestrial	1982	23.73	1,791.69	
3	Star Wars: The Force Awakens	2015	184.00	1,902.72	
8094	The Net	1995	31.48	1,583.05	
10110	One Hundred and One Dalmatians	1961	29.18	1,574.81	
7309	The Empire Strikes Back	1980	47.63	1,424.63	

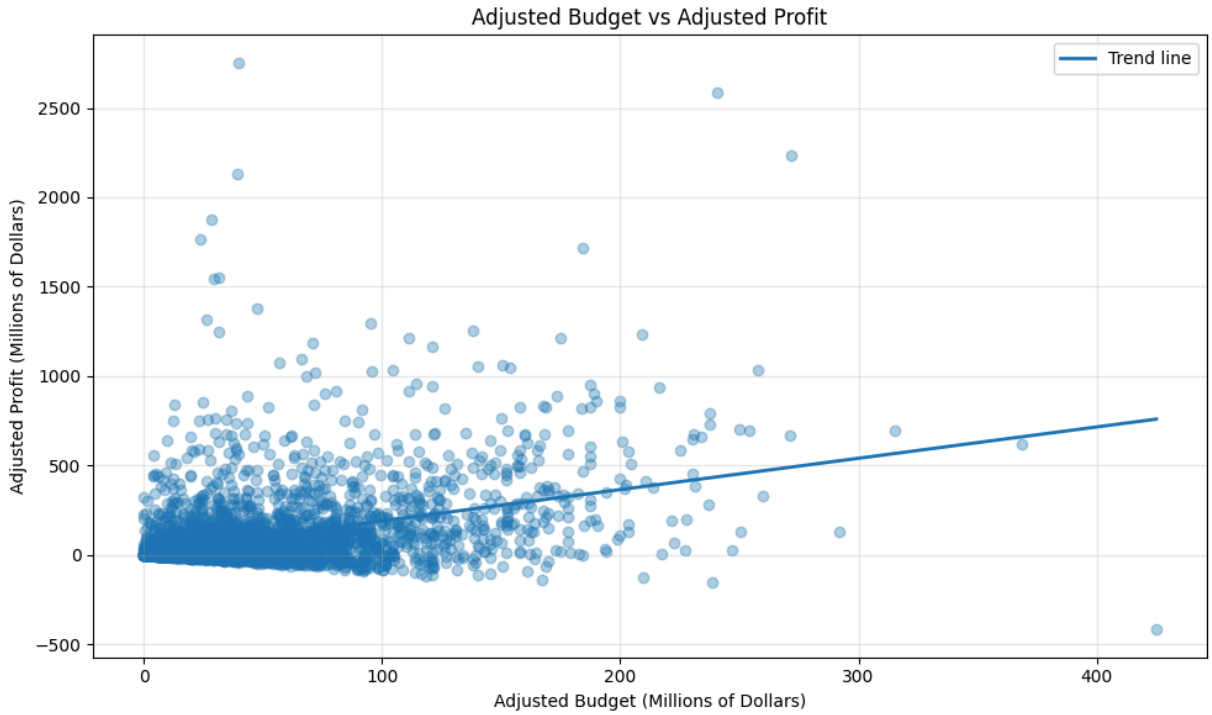
The adjusted profit distribution is wide and skewed. The median adjusted profit is much lower than the mean adjusted profit, which shows that a small number of very high-profit movies pull the average upward. About 72% of the movies in the profit dataset have positive adjusted profit.

Research Question 2: Do higher adjusted budgets relate to higher adjusted profits?

Next, I will compare adjusted budget to adjusted profit. This helps show whether more expensive movies tend to earn more profit after adjusting for inflation.

In [18]:

```
scatter_with_trend(
    profit_movies,
    'budget_adj_millions',
    'profit_adj_millions',
    'Adjusted Budget vs Adjusted Profit',
    'Adjusted Budget (Millions of Dollars)',
    'Adjusted Profit (Millions of Dollars)'
)
```



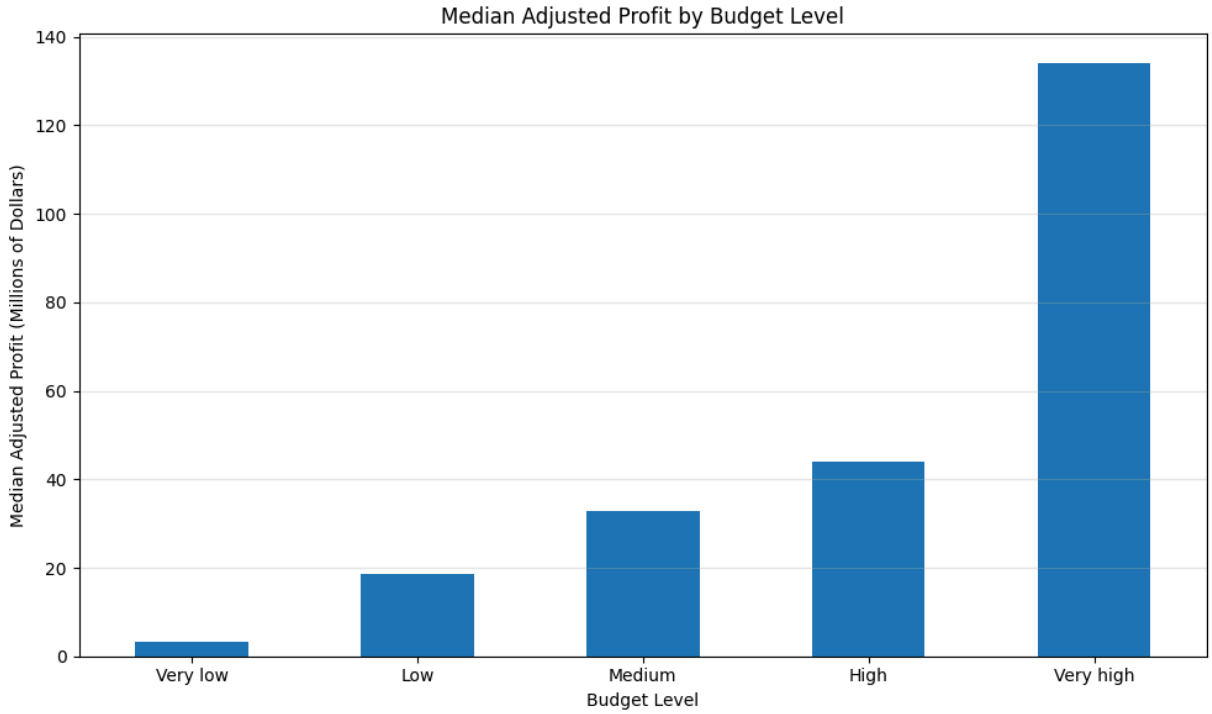
In [19]:
```
# Create budget groups using quintiles
budget_labels = ['Very low', 'Low', 'Medium', 'High', 'Very high']
profit_movies['budget_level'] = pd.qcut(
    profit_movies['budget_adj'],
    q=5,
    labels=budget_labels
)

budget_summary = summarize_profit_by_category(profit_movies, 'budget_level')
budget_summary
```

Out[19]:

budget_level	movie_count	mean_profit_millions	median_profit_millions	profitable_ra
Very low	773	25.63	3.37	0.6
Low	769	46.24	18.74	0.1
Medium	770	79.81	32.73	0.
High	774	100.07	43.93	0.1
Very high	768	212.85	133.95	0.

In [20]:
```
# Bar chart of median adjusted profit by budget level
budget_summary['median_profit_millions'].plot(kind='bar', figsize=(10, 6))
plt.title('Median Adjusted Profit by Budget Level')
plt.xlabel('Budget Level')
plt.ylabel('Median Adjusted Profit (Millions of Dollars)')
plt.xticks(rotation=0)
plt.grid(True, axis='y', alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



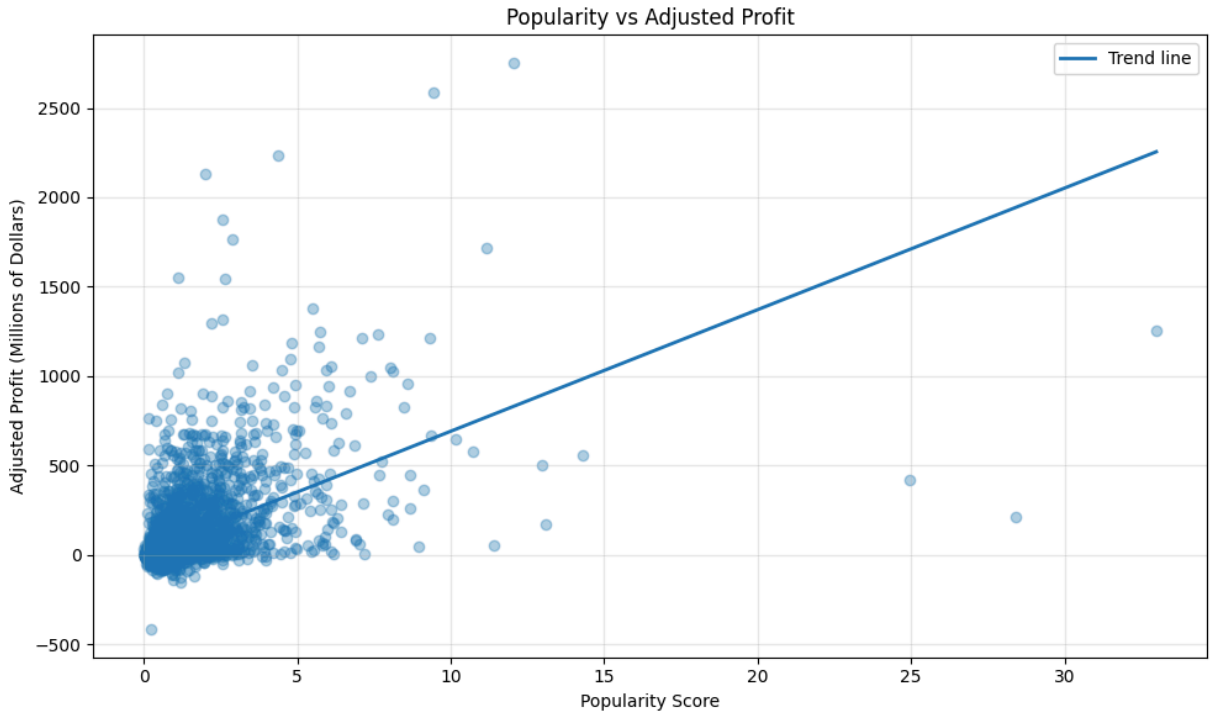
The scatter plot shows a positive relationship between adjusted budget and adjusted profit, but there is also a lot of variation. Some high-budget movies earned very large profits, while others lost money.

The budget group summary gives a clearer view. Movies in the very high budget group have the highest median adjusted profit and the highest profitable rate. This suggests that larger budgets are associated with stronger profits in this dataset, but a higher budget does not guarantee a profitable movie.

Research Question 3: How do popularity and user engagement relate to adjusted profit?

This section compares adjusted profit with popularity, vote count, and vote average. These variables represent different forms of audience attention or audience response.

In [21]:
```
scatter_with_trend(
    profit_movies,
    'popularity',
    'profit_adj_millions',
    'Popularity vs Adjusted Profit',
    'Popularity Score',
    'Adjusted Profit (Millions of Dollars)'
)
```



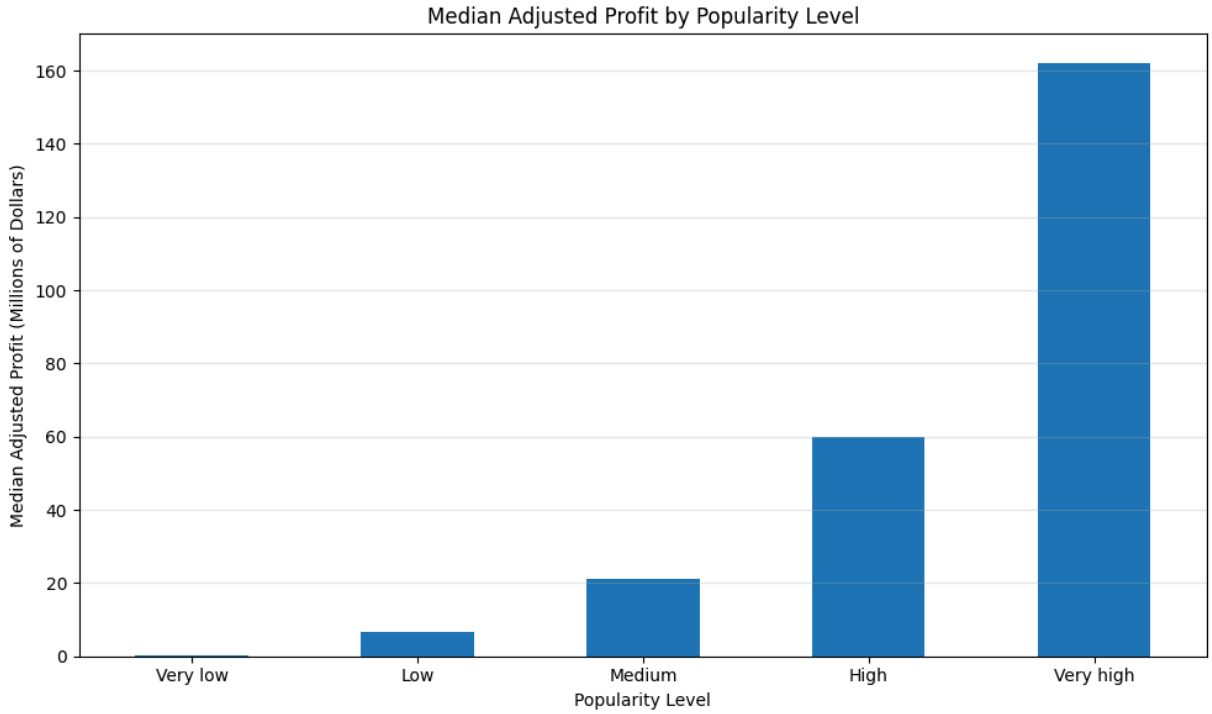
In [22]:
```
# Create popularity groups using quintiles
popularity_labels = ['Very low', 'Low', 'Medium', 'High', 'Very high']
profit_movies['popularity_level'] = pd.qcut(
    profit_movies['popularity'],
    q=5,
    labels=popularity_labels
)

popularity_summary = summarize_profit_by_category(profit_movies, 'popularity
popularity_summary
```

Out[22]:

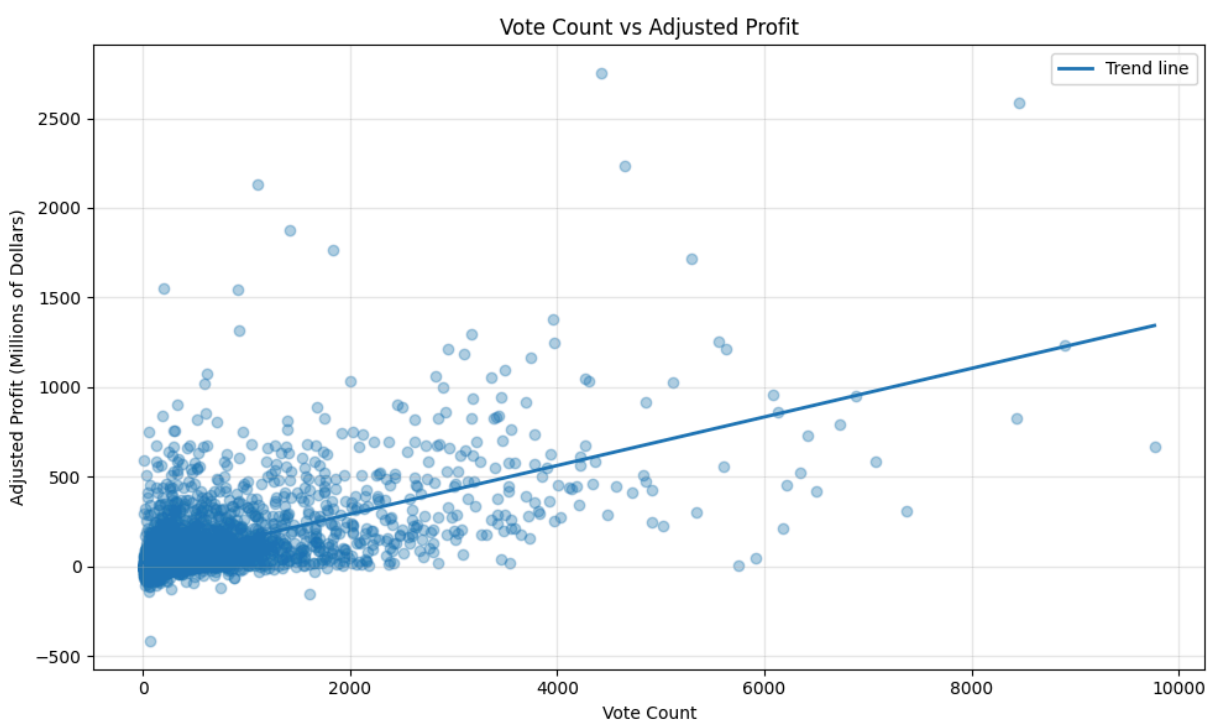
popularity_level	movie_count	mean_profit_millions	median_profit_millions	profitable
Very low	771	15.51	0.10	
Low	771	26.69	6.78	
Medium	770	52.29	21.01	
High	771	103.08	59.80	
Very high	771	266.51	161.92	

In [23]:
```
# Bar chart of median adjusted profit by popularity level
popularity_summary['median_profit_millions'].plot(kind='bar', figsize=(10, 6
plt.title('Median Adjusted Profit by Popularity Level')
plt.xlabel('Popularity Level')
plt.ylabel('Median Adjusted Profit (Millions of Dollars)')
plt.xticks(rotation=0)
plt.grid(True, axis='y', alpha=0.3)
```

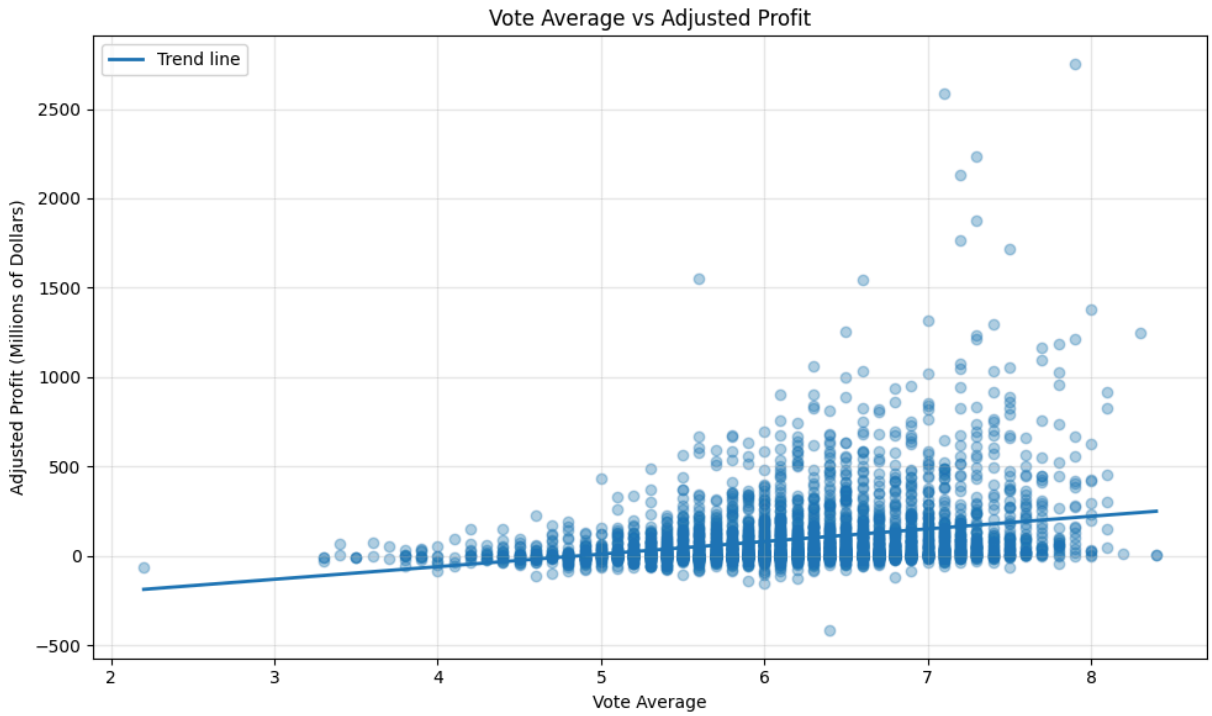
```
plt.tight_layout()
plt.show()
```



In [24]:
```
scatter_with_trend(
    profit_movies,
    'vote_count',
    'profit_adj_millions',
    'Vote Count vs Adjusted Profit',
    'Vote Count',
    'Adjusted Profit (Millions of Dollars)'
)
```



In [25]:
```
scatter_with_trend(
    profit_movies,
    'vote_average',
    'profit_adj_millions',
    'Vote Average vs Adjusted Profit',
    'Vote Average',
    'Adjusted Profit (Millions of Dollars)'
)
```



In [26]:
```
# Correlation table for adjusted profit and selected numeric variables
correlation_columns = [
    'profit_adj', 'budget_adj', 'popularity', 'vote_count', 'vote_average'
]

profit_correlations = profit_movies[correlation_columns].corr(numeric_only=T
profit_correlations
```

Out[26]:
```
profit_adj     1.00
vote_count     0.61
popularity     0.52
budget_adj     0.40
vote_average   0.29
Name: profit_adj, dtype: float64
```

Popularity and vote count both show a positive relationship with adjusted profit. This makes sense because more profitable movies are often watched by more people, talked about more, and receive more votes.

Vote average also has a positive relationship with adjusted profit, but the relationship is weaker than vote count and popularity. This suggests that the amount of audience engagement may be more strongly associated with profit than the average rating alone.

The correlation table supports this pattern. Vote count and popularity have stronger positive correlations with adjusted profit than vote average.

Conclusions

The main goal of this project was to investigate which movie characteristics are associated with higher adjusted profit in the TMDb movie dataset.

The analysis found that adjusted profit varies widely. The median adjusted profit is much lower than the mean adjusted profit because a small number of blockbuster movies earned extremely large profits. About 72% of movies in the cleaned profit dataset had positive adjusted profit.

Higher adjusted budgets are associated with higher adjusted profits, especially when movies are grouped into budget levels. The very high budget group had the highest median adjusted profit and the highest profitable rate. However, some high-budget movies still lost money, so budget alone does not guarantee success.

Popularity and vote count showed stronger positive relationships with adjusted profit than vote average. This suggests that audience attention and engagement are more closely connected to profit than rating score alone. Vote average still had a positive relationship, but it was weaker.

Overall, the variables most associated with higher adjusted profit in this analysis were adjusted budget, popularity, and vote count. These findings show relationships in the data, but they do not prove causation.

Additional Research

Additional research could improve this analysis by including more complete financial data, especially marketing costs, distribution costs, streaming revenue, international box office revenue, and licensing revenue. This would make the profit calculation more complete. Future analysis could also compare genres, production companies, and release periods to see whether certain types of movies are more consistently profitable.

Limitations

There are several limitations in this analysis.

First, many movies in the original dataset had zero values for budget or revenue. These values likely represent missing financial data instead of true zero-dollar budgets or revenues. Because adjusted profit depends on both budget and revenue, those rows had to be removed from the profit analysis. This reduced the dataset from 10,866 movies to

3,854 movies, which may create bias if the remaining movies are mostly larger or better-documented releases.

Second, the dataset does not include every cost or revenue source that affects real movie profit. Marketing costs, distribution costs, streaming revenue, international release differences, and later licensing deals are not included. Because of this, `revenue_adj - budget_adj` is only an estimate of profit, not a complete business profit calculation.

Third, the popularity score comes from TMDb and may be influenced by user activity on that platform. It may not perfectly represent general audience popularity across all countries, theaters, or time periods.

Finally, this project is exploratory and does not use statistical tests or controlled experiments. The results describe patterns and correlations, not cause-and-effect relationships.

In [27]:
```
# Running this cell will execute a bash command to convert this notebook to
!python -m nbconvert --to html Investigate_a_Dataset.ipynb
```

```
[NbConvertApp] Converting notebook Investigate_a_Dataset.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 7 image(s).
[NbConvertApp] Writing 950098 bytes to Investigate_a_Dataset.html
```